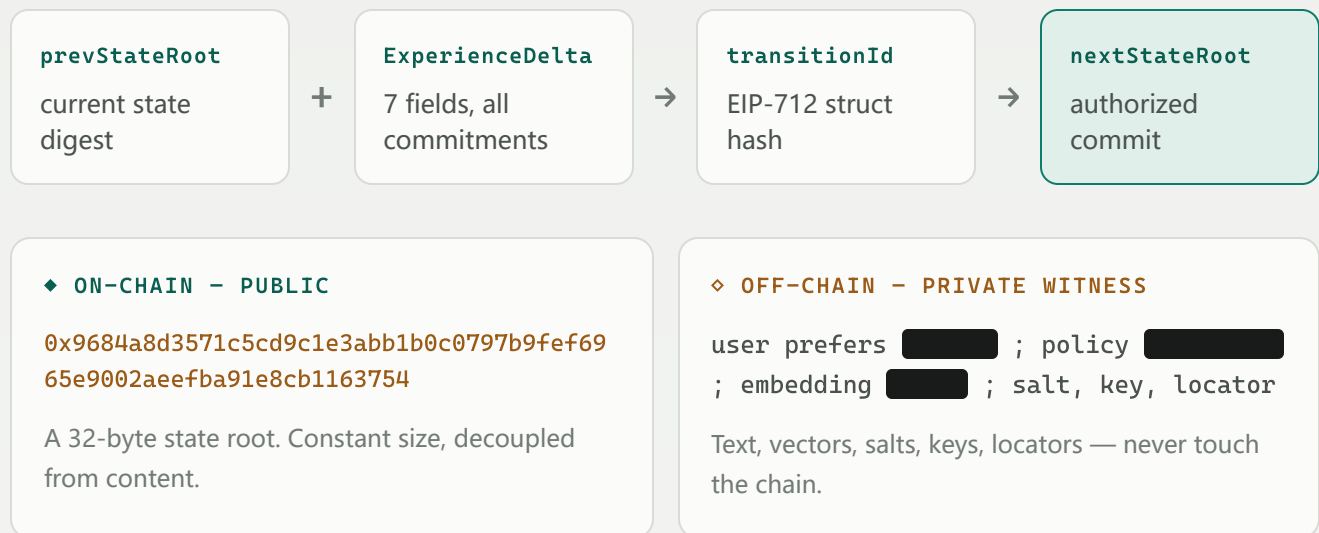


A verifiable, private state layer for agent memory

Commit the proof of a memory transition — never the memory itself.

Memory is the new state, but raw cognition should never be public calldata.

记忆是新的状态，但原始认知永远不该成为公开的调用数据。



AUTHOR

Everest An · @everest-an

STATUS

Community draft (pre-ERC)

REFERENCE RELEASE

1.0.0-alpha.1 · 2026-07-14

LICENSE

CC0 · public domain

§1 Introduction — from “state” to “memory state”

Bitcoin gave value a trustless state. Ethereum gave any computable state a trustless transition. As computation’s protagonist becomes the autonomous agent, a new and most-important kind of state emerges: memory.

1.1 Blockchains as state transition systems

A ledger can be read as a **state transition system**: a state (who owns what) and a function that takes the current state and a transaction and produces the next — `APPLY(S, TX) → S'`, or `ERROR`. Bitcoin’s state is the UTXO set; Ethereum generalized it to accounts and made the function Turing-complete. The power is not record-keeping — it is that **anyone can independently replay history and verify the current state without trusting any single party**. “What the next valid state is” moved from a central authority to deterministic rules everyone can run.

1.2 The new state: agent memory

A valuable agent is not a stateless function. It remembers interactions and forms preferences; distills reusable skills from repeated tasks; accumulates knowledge into retrievable cards; evolves its world model over time. This

memory *is* the agent's identity and its asset. An assistant that has learned your habits over three years and a freshly initialized blank model are two entirely different economic entities.

Memory is an agent's on-chain balance — it determines what the agent is worth, who can trust it, to whom it can be sold. Yet today it lives in isolated black boxes. You cannot **prove** an unaltered learning history, **carry** memory across platforms without leaking it, **audit** when and by whom it was written, or let cooperating agents reach trustless consensus on shared memory.

1.3 Thesis

THE WHOLE DESIGN TURNS ON ONE SENTENCE

The opportunity: turn agent memory into a verifiable state machine and you have the trust layer for the agent economy. The constraint: memory content is intensely private — it must never enter public calldata. The elegance is satisfying both at once — **on-chain goes the proof that a legitimate, ordered, authorized transition occurred; not the memory.**

1.4 What a bare content hash fails to establish

A hash alone does not establish: **(1)** which state it advances; **(2)** whether it is the unique successor; **(3)** who controls the namespace and may authorize updates; **(4)** whether all meaningful references were signed; **(5)** whether independent implementations derive the same identifier. Agent Memory State v1 answers all five in one interoperable state machine.

§2 Design principles

Five principles constrain one another and together define what a *correct* implementation is.

PRINCIPLE	WHAT IT MEANS
Privacy first	The chain sees only <i>commitments</i> — salted/encrypted hashes — never plaintext text, embeddings, preferences, policies, latent state, salts, keys, or locators. Not optional: raw cognition in calldata is non-conforming.
Determinism	The same delta yields a bit-for-bit identical <code>transitionId</code> in Solidity, TypeScript, or any client — enforced by fixed type strings, fixed ABI encoding, and golden vectors.
Authorization is cryptographic	Write permission is bound to a space’s initial controller: <code>spaceId</code> derives from controller + salt, so a namespace claim is specific to whoever holds the key.
Signer-agnostic	EOA (non-malleable ECDSA), smart / multisig / abstract account (EIP-1271), or the controller calling directly (empty signature).
Attack-resistant commitments	Commitments to low-entropy content are brute-forceable, so they must be salted or encrypted. Real locators stay a private witness.

§3 The protocol: Agent Memory State v1

We follow the Ethereum whitepaper’s order — the “account,” the “transaction,” the “state transition function,” then “authorization.” The mapping to concepts you know:

ETHEREUM	AGENT MEMORY STATE	MEANING
Account	<code>MemorySpace</code>	Container of state and identity
Transaction	<code>ExperienceDelta</code>	Atomic operation advancing state
State root	<code>stateRoot</code>	Cryptographic digest of current state
Tx hash	<code>transitionId</code>	Unique identifier of one operation
nonce / order	<code>sequence</code>	Anti-replay, strict ordering
Signature	<code>authorizer + EIP-712</code>	Who may advance state

The crucial difference: an Ethereum transition writes *how much moved* into public state; an Agent Memory State transition writes only the *proof a legitimate transition occurred* — content stays off-chain.

3.1 MemorySpace — identity & container

A memory space holds one linear history, analogous to an account. Its identity is derived, not assigned:

```
MEMORY_SPACE_TYPE = "MemorySpace(address initialController,bytes32 salt)"

spaceId = keccak256(abi.encode(
    keccak256(bytes(MEMORY_SPACE_TYPE)),
    initialController,
    salt
))
```

The claim “I create this namespace” is thus cryptographically specific to `initialController` — nobody without the key can forge a `spaceId` pointing at that controller. Each space keeps a **head** (latest `sequence` ,

current `stateRoot`) under a **controller** + **authorizer** model with nonce rotation. The controller is the recovery boundary; the authorizer approves transitions and may be a hot EOA, multisig, smart account, or policy contract. Rotation preserves identity and history.

3.2 ExperienceDelta — the atomic operation

One evolution of memory is abstracted as an experience delta. The normative struct has exactly **seven** fields — no more, no less:

```
struct ExperienceDelta {  
    bytes32 spaceId;           // which memory space  
    uint64  sequence;         // strictly increasing, begins at 1  
    bytes32 prevStateRoot;    // state root before this transition (0 for first)  
    bytes32 deltaCommitment;   // commitment to *what changed* (non-zero)  
    bytes32 provenanceCommitment; // commitment to *where it came from* (0 = absent)  
    bytes32 profileId;         // interpretation profile (non-zero)  
    bytes32 locatorCommitment; // commitment to the off-chain locator (0 = absent)  
}
```

Deliberately austere: the spec **excludes** author, timestamp, raw URI, memory-type enum, previous-delta pointer, and separate input/inference fields from the transition. Every extra field admitted into the hash is one more surface for cross-implementation drift and one more channel for leakage. Seven is the minimal set *sufficient* to express a meaningful, authorizable, auditable transition while *revealing* no cognitive content — three commitment fields (what changed / where from / where it lives) carry all of “privacy first.”

3.3 The state transition function

Step 1 — compute the transitionId from the fixed type string and ABI encoding:

```
transitionId = keccak256(abi.encode(  
    keccak256(bytes(EXPERIENCE_DELTA_TYPE)),  
    spaceId, sequence, prevStateRoot, deltaCommitment,  
    provenanceCommitment, profileId, locatorCommitment  
))
```

Because the type string and encoding are fully fixed, every conforming implementation computes the same value. There is no alternate JSON, JCS, CBOR, or app-specific ID.

Step 2 — fold the next state root, chaining the previous root and this transition:

```
nextStateRoot = keccak256(abi.encode(  
    keccak256(bytes(MEMORY_STATE_TYPE)),  
    prevStateRoot, transitionId  
))
```

The history becomes an **append-only state accumulator**: every `stateRoot` commits to the whole sequence from genesis; tampering with any past transition invalidates every later root — exactly as altering an old block does. Critically, **the registry computes `nextStateRoot`**; callers cannot supply it.

3.4 Authorization & consensus

A valid root is necessary, not sufficient. For the registry to **accept** a transition: the space exists; `deltaCommitment` and `profileId` are non-zero; `sequence == head.sequence + 1`; `prevStateRoot == head.stateRoot`; and the authorizer approves. Authorization is an EIP-712 signature over a fixed domain:

```
name           = "AgentMemoryState"
version        = "1"
chainId        = current chain ID
verifyingContract = registry address
```

The fixed domain closes cross-chain and cross-registry replay. Signers: EOAs (non-malleable ECDSA), smart/multisig accounts (EIP-1271, `0x1626ba7e` magic value), or the controller directly (empty signature only when `msg.sender` is the required account). The key rule: **signatures cover all delta fields, including `locatorCommitment`** — which is exactly what closes the relay-manipulation gap. A relayer cannot substitute the locator without invalidating the signature.

3.5 Baseline private commitments

Profiles may define stronger schemes (including zero-knowledge); a conforming implementation should support this domain-separated baseline:

```
deltaCommitment      = keccak256(abi.encode(DELTA_DOMAIN, profileId, deltaSalt, keccak256(
provenanceCommitment = keccak256(abi.encode(PROVENANCE_DOMAIN, provenanceSalt, keccak256(
locatorCommitment    = keccak256(abi.encode(LOCATOR_DOMAIN, locatorSalt, keccak256(
```


Each salt is 32 bytes, independently sampled. For low-entropy plaintext the salt **must** stay secret, or the payload **must** be ciphertext from a fresh high-entropy key and nonce — a public salt does not stop targeted dictionary attacks.

§4 Security considerations

A protocol carrying real value must draw its security boundary honestly. This states what it protects, what it trusts, and — crucially — what it does *not* promise.

§4.1 · THE CRITICAL NON-CLAIM

A valid transition proves only that a configured authority approved a particular committed state transition. It does *not* prove the truthfulness of the content, the availability of off-chain data, any ownership, or that a deletion occurred. The registry verifies exactly two things — **authorization** and **state-machine continuity** — and does not pretend to know more.

4.2 Trust assumptions

The registry is trusted only to execute its published bytecode. Storage providers, relayers, indexers, and profile publishers get *no* implicit trust. This minimal-trust stance lets the security be argued mechanically.

4.3 Threats, mitigations, residual risk

THREAT	MITIGATION	RESIDUAL RISK
Namespace hijacking	<code>spaceId=keccak(controller,salt) ;</code> registration needs authorization	Controller key compromised before registration
Unauthorized modification	Registry honors only the configured authorizer	Permissive or compromised authorizer
Relay manipulation	Every field (incl. <code>locatorCommitment</code>) is signed	Relayer can censor/delay, not modify
Invalid prior state	Exact <code>prevStateRoot</code> + <code>sequence</code> check	Linear model forbids concurrent branches
Cross-chain replay	EIP-712 <code>chainId</code> binding	Identical-parameter forks need ops response
Config replay	Per-space <code>configNonce</code> rotation	Controller compromise → malicious update
Locator exposure	Only a salted commitment is public	Out-of-band witness disclosure
Hash drift	Golden vectors across Solidity / TS	New impls may skip conformance

4.4 The cryptography of private commitments

Hashing short memory directly is brute-forceable — “user prefers dark mode” can be enumerated and matched. Hence 32-byte domain-separated salts, and for low-entropy payloads a secret salt or encryption with a

random key and nonce. Salts, keys, plaintext, and locators form the **private witness**, entirely off-chain. This shifts a clear responsibility onto applications — **they bear witness preservation and key recovery** — and leaves honest side channels: transition *timing*, *frequency*, and *profileId* remain visible.

4.5 Pre-deployment gates

Seven gates before production: EIP-712 validation, fuzzing the sequence logic, cross-verification by an independent implementation, encryption-profile review, recovery-procedure drills, testnet monitoring, and an external Solidity audit. Security is a set of executable checkpoints, not a one-time declaration.

§5 Applications & future work

The core does only “authorization + continuity.” Product value grows in the layers above it — and that layering is itself the key architectural decision.

5.1 Four-layer architecture

Early prototypes bundled state primitives, memory frameworks, deletion, market settlement, a “REC language,” and agent behaviors together — too coupled to audit. The refactor split them into four tiers, each with its own trust model:

Adapters

Awareness taxonomy, payload construction, real-memory glue

INTEGRATION

Experimental

Memory Market, REC / cognitive assets

VISION

Extensions

DeletionAttestation, etc. — each states its own trust model

OPT-IN

Core

Authorization · delta hashing · state rules · signatures · events

NORMATIVE

The core is small enough for anyone to implement and audit independently; products evolve above it without touching the base.

Ethereum's "thin protocol, thick applications" applied to memory — markets, deletion, and cognitive assets live outside the core, just as ERC-20 and NFTs live outside Ethereum's.

5.2 Extensions — deletion attestations

The "right to be forgotten" is unavoidable, but on an append-only accumulator deletion is a *claim*, not a physical fact. A deletion-attestation extension can publish an "I deleted X" commitment, but it **must state its own trust model**: the chain proves an authority *attested* to deletion, never that off-chain data was truly erased. Making the limit explicit avoids the misleading promise of "on-chain deletion."

5.3 Experimental — the Memory Market

`MemoryMarket.sol` is the first economic-layer prototype: a **time-limited licensing marketplace for memory spaces**. What is traded is a time-bounded license to access a specific committed state — *not* the plaintext.

- **List** — the controller sets token, price, duration, optional royalty (0–10,000 bps), and **captures the current state root** — anchoring the license to an exact configuration to prevent "stale-state sales."

- **Purchase** — reentrancy-guarded; validates the listing is active *and the state root is unchanged*; splits payment between royalty recipient and seller; extends expiry **additively** (stacking, not resetting).
- **Views** — `hasLicense()`, `licenseExpiresAt()`, `getListing()` for composable queries.

The market settles in an **arbitrary ERC-20**. The core binds no native token; an ecosystem settlement / governance token (such as **AWAR**) is a natural evolution of this layer, *not a prerequisite* of the protocol — preserving “neutral core, optional economics.”

5.4 Experimental — cognitive assets & REC (vision)

Further out: capitalize memory into **cognitive assets** and a **REC** vocabulary for cognitive value and licensing. In honesty: as of `1.0.0-alpha.1` these have **no contract code** — the architecture record marks them “product-level evolution, not foundational protocol.” Keeping them experimental and isolated is how the vision is kept from contaminating the auditable core.

5.5 Adapters — connecting a real memory system

`@erc-awar/awareness-adapter` converts Awareness knowledge cards and embeddings into a public `ExperienceDelta` plus a private witness. Card type maps to an interpretation profile:

CARD TYPE	PROFILE	SEMANTICS
decision	POLICY	Constrains the agent's future behavior
insight / solution / workflow	TEXT	Human-readable knowledge
memory	EPISODIC	A recollection observed at a point in time
task	EPISODIC	A unit of work tracked over time

The full profile vocabulary (`TEXT`, `EMBEDDING`, `LATENT`, `TOOL_TRACE`, `EPISODIC`, `POLICY`, `SHARED_WORKING`, `PROOF`) is adapter-owned — *not* protocol enums — so it evolves freely. The adapter is deliberately thin: it does **not** generate salts or construct commitments — those are delegated to the reference `MemoryStateMachine`. A real memory system integrates by mapping records into cards and providing salts and locators; the adapter cleanly splits the **public delta** (safe for calldata) from the **private witness** (locators + salts, off-chain).

5.6 Migration

Migrating from v0 means creating a **new v1 space** with a private migration payload documenting the v0 export, identifiers, tooling, and audit; the original chain is untouched. v0's JCS identifiers, unsigned URIs, and fixed enums are explicitly **not** wire-compatible — a clean break beats false compatibility carrying baggage.

§6 Miscellanea & concerns

- **Linear vs concurrent.** Strict linear advancement buys anti-replay/reorder/fork guarantees at the cost of no concurrent branches within a space. Concurrency is served by multiple spaces + off-chain merge — a conscious trade of flexibility for auditability.
 - **Gas & scalability.** Each transition writes only a few `bytes32` — fixed-width, decoupled from content size. One sentence or an entire vector store, the on-chain footprint is constant; a natural fit for L2s and rollups.
 - **Availability is the app's job.** `locatorCommitment` commits to *what* the locator is, not that its content still exists. Availability, backup, witness preservation, and key recovery are application responsibilities — an honest boundary, not a defect.
 - **Profiles, not enums.** The core freezes no taxonomy; `profileId` is application-defined — as Ethereum left “token” to ERC-20.
 - **Road to a formal ERC.** A Magicians thread, discussion URL in metadata, confirmed authors and an EIP champion, cryptography/agent reviewers, an independent second implementation passing the golden vectors, an external audit and testnet validation, then a PR to `ethereum/ERCs`. Standardization is public and multi-party — not a unilateral declaration.
-

§7 Conclusion

Agent Memory State v1 gives memory what it needs: a deterministic, privacy-first, strictly authorized, cross-implementation-interoperable state machine. Its whole insight is one restrained choice — **commit the proof of a memory-state transition, not the memory itself**. A seven-field delta, a bit-for-bit `transitionId`, an append-only accumulator, and an honestly drawn trust boundary form a foundation that can be independently audited, freely extended, and relied upon by real value.

On that foundation, memory can — for the first time — become a verifiable, portable, tradable asset without losing its privacy: deletion proven *as attested*, access *time-licensed*, cognition *capitalized*. The core stays neutral and small; products bloom above it.

IN ONE LINE

Memory is the new state. AwareLiquid makes that state, for the first time, both verifiable and private. This is the starting point for the trust layer of the agent economy.

§8 Open questions — request for feedback

Feedback is invited on the Ethereum Magicians thread. The questions we most want the community's input on:

1. **Linearity.** Is a strictly linear per-space state machine the right base primitive, or should the core admit a minimal branch/merge notion (at the cost of auditability)?
 2. **Baseline commitments.** Is the domain-separated keccak baseline an acceptable normative floor, with ZK schemes as opt-in profiles? Should the ERC mandate encryption for any low-entropy payload rather than merely a secret salt?
 3. **EIP-1271 edge cases.** Which authorizer-contract behaviors (reverts, malformed returns, `STATICCALL` gas griefing) should the spec call out more forcefully?
 4. **profileId governance.** Should there be any registry-level convention for profile identifiers, or is fully application-defined the right long-term stance?
 5. **Deletion semantics.** Is “attestation with a stated scope” the right framing, and should the core reference it normatively or stay silent?
 6. **Observation time.** Is recording `block.timestamp` as observation time (never as ordering input) sufficient?
 7. **Naming.** Is “Agent Memory State” the right title, or should it lead with “Commitments” / “Registry” to set expectations about the non-claim?
-

§A Appendix — canonical constants & golden vectors

Golden typehashes (canonical v1 vector):

ExperienceDelta typehash:

0x4f020f86bc06d852f1fde17853b4d92a70214eeab8e09718028124af097d070d

MemoryState typehash:

0xf3148762556cbf851baf4b9a205e18ff4e6b366a58a3a1ef58e8626ba41beadb

MemorySpace typehash:

0x9ae5478f084ad3b841da58a9cb2354d153cddec59ee64d0cb741fa9d08884531

Golden derived values:

transitionId:

0xdd00dd6eb3aec704b5455502647a0caacf23be6c724eda4a60d9645291e7f4e5

nextStateRoot:

0x9684a8d3571c5cd9c1e3abb1b0c0797b9fef6965e9002aeefba91e8cb1163754

Verification:

One-shot: install + full check

pnpm install --frozen-lockfile && pnpm check

Run separately

pnpm test:ts

cd contracts && forge fmt --check && forge test

Any implementation must reproduce the three typehashes and every transition ID bit-for-bit against the cross-language `test-vectors/v1.json`.

Companion to ERC-XXXX · [erc/erc-xxxx-agent-memory-state.md](#) · Repository

[AwareLiquid/ERC-AWAR](#) · Copyright waived via CC0. This whitepaper is a community draft. Some features described (Memory Market, cognitive assets, REC) are experimental or aspirational and constitute neither investment advice nor a commitment to future functionality. The protocol core's security boundary is governed by the non-claim in §4.1.